neural-network-benchmark

Cómo funciona el proyecto, módulo por módulo — y cómo se relacionan entre sí

Explicación en lenguaje natural · Paquete `prebenchmark/` · Fase de pre-benchmark

---

Índice

1. Qué es este proyecto, en una frase
2. El punto de partida: las scripts legadas en `Experimentos/`
3. El recorrido general: el pipeline de 5 etapas
4. Los módulos, uno por uno (4.1–4.14)
5. Cómo se relacionan los módulos (mapa mental)
6. Las ideas que sostienen todo el diseño
7. Decisiones arquitectónicas y consideraciones generales
    - 7.1 Protocolo multi-semilla: cómo medimos la estabilidad
    - 7.2 Comparación controlada vs. multi-variable ("solo informativo")
    - 7.3 MLP entrenado por retropropagación, no el perceptrón clásico
    - 7.4 El criterio de corte (early stopping)
    - 7.5 Guardar los pesos de la mejor época (`restore_best_weights`)
8. Resultados de los análisis obtenidos
    - 8.1 El early stopping cambió por completo el costo del barrido
    - 8.2 ¿En cuántas épocas convergen la mayoría de las combinaciones?
    - 8.3 La familia `lr = 0.001` + `sgd` : un problema de convergencia
    - 8.4 Por qué bajamos el tope por defecto a 500 épocas
    - 8.5 CPU vs. GPU: por qué la GPU no aportaba mejoras
    - 8.6 Resumen de tiempos
9. Cómo registramos los resultados y cómo hacemos el análisis
    - 9.1 ¿Dónde queda todo?
    - 9.2 Dentro de cada carpeta de corrida (`<run_id>/`)
    - 9.3 Los archivos en `results/exports/<experiment_id>/`
    - 9.4 Cómo leerlo sin cargar todo en memoria
10. Cómo interaccionar con el sistema (CLI)
    - 10.1 Punto de entrada, opciones comunes y códigos de salida
    - 10.2 `generate` : registrar la grilla de candidatos
    - 10.3 `run` : ejecutar configuraciones
    - 10.4 `compare` : comparar contra una línea base
    - 10.5 `aggregate` : agrupar repeticiones
    - 10.6 `export` : generar las tablas CSV
    - 10.7 Flujos completos de ejemplo
    - 10.8 Errores frecuentes y cómo resolverlos
    - 10.9 Referencia rápida

Este documento explica, con palabras y no solo con código, **qué hace cada módulo del proyecto**, **qué aporta al funcionamiento general** y **cómo se conecta con los demás**. La idea es que cualquier persona pueda entender el sistema completo de arriba hacia abajo: primero el propósito, después el recorrido de los datos, y finalmente el detalle de cada pieza.

1. Qué es este proyecto, en una frase

El proyecto es una **infraestructura experimental** para comparar redes neuronales de forma ordenada y trazable. No es "una red neuronal": es el **laboratorio** alrededor de la red. Su trabajo es **registrar** una grilla de configuraciones candidatas, **ejecutarlas** de manera reproducible, **guardar** cada resultado sin poder sobreescribirlo, **compararlas** contra una línea base, **agrupar** repeticiones para medir su variabilidad, y **exportar** todo a tablas CSV listas para analizar. La meta de fondo es que cada afirmación ("esta configuración es mejor") pueda rastrearse hasta el **exacto** conjunto de parámetros y la **exacta** versión del dataset que la produjeron.

El nombre `prebenchmark` ("pre-benchmark") refleja su rol: es la etapa previa al benchmark formal. Primero se explora y se valida la maquinaria; después vendrá la comparación grande.

2. El punto de partida: las scripts legadas en `Experimentos/`

Antes de este paquete, todo el trabajo vivía en tres scripts monolíticos dentro de `Experimentos/` (`wine.py`, `banknote.py`, `wine_quality.py`). Cada script repetía el mismo pipeline de punta a punta: fijar semilla → dividir datos → escalar → armar el MLP → inicializar pesos → entrenar → medir métricas → imprimir en pantalla.

Ese enfoque funcionaba, pero tenía tres problemas serios: **(1)** los resultados se **imprimían** y nunca se guardaban de forma estructurada; **(2)** la lógica estaba **duplicada** en cada script; y **(3)** no había forma de comparar configuraciones ni de registrar una decisión con su justificación. La decisión de diseño fue **no tocar** esos scripts (quedan como referencia histórica) y **extraer** el pipeline a un paquete nuevo, generalizándolo. Esa es la razón de que exista `prebenchmark/`: no se importa desde lo legado, se **reconstruye mejor**.

3. El recorrido general: el pipeline de 5 etapas

Todo el paquete gira alrededor de un flujo que se ejecuta en orden, y cada etapa deja un rastro en disco. Se puede pensar como una línea de montaje:

```
  register ──► run ──► compare ──► aggregate ──► export
   (definir      (entrenar    (comparar    (agrupar       (tablas
    la grilla)    cada config)  vs baseline) repeticiones)  CSV)
```

En términos del código, el flujo de datos es este:

```
ParameterSpace ──generate_configs──► [36 ExperimentConfig]
       │
       └── validate_config ──► VALID / WARNING (ejecutable) | INVALID (bloqueado)
                                        │
   run_experiment ── new_run_id ──► bucle de entrenamiento ──► RunResult
       │   data.load_wine_quality → build_splits → build_dataloaders
       │   build_model → initialize_weights → optimizador → métricas
       └──────────────────────► store.persist_run (una sola escritura)
                                        │
        compare (baseline + guardas) ──┤
        aggregate (agrupar por config)─┤
        decisions (log manual) ────────┤
                                       ▼
                    export.export_all → results/exports/<experimento>/
```

4. Los módulos, uno por uno

Cada módulo tiene una responsabilidad clara y un "contrato" (su API pública). La regla de oro del diseño es que **cada módulo hace una cosa** y que las capas puras (config, space, compare, aggregate, decisions) **no importan** torch ni hacen entrada/salida, por lo que son seguras de importar en cualquier lado y fáciles de testear.

4.1 `config.py` — el plano de la obra núcleo

Qué hace. Define el **esquema** de una configuración de experimento usando *dataclasses* tipadas: una clase `ExperimentConfig` con todos los parámetros, y sub-especificaciones anidadas (`DatasetSpec` , `ArchitectureSpec` , `SplitSpec` , `PreprocessingSpec` , `ConvergenceSpec`).

Qué aporta. Es el **contrato de datos** de todo el sistema. Aquí viven los valores por defecto del protocolo (split 60/20/20, batch 32, 500 épocas, semilla 42) y las **constantes de la línea base** (ReLU + Adam + lr 0.001 + He). También guarda los **registros de nombres válidos** (`KNOWN_ACTIVATIONS` , `KNOWN_OPTIMIZERS` , etc.) que serán la única fuente de verdad para validar. Incluye `to_dict()` / `from_dict()` con orden de campos **canónico** (clave para calcular huellas de identidad), y `resolve_convergence()` , que rellena con valores documentados los campos de convergencia que el usuario deje vacíos.

Con quién se relaciona. Es la base de casi todo: `space.py` la usa para generar la grilla, `identity.py` para calcular hashes, `runner.py` para entrenar, y `store.py` para persistir el snapshot. No importa librerías de terceros: es puramente declarativo.

4.2 `space.py` — la grilla de candidatos y su validación núcleo

Qué hace. Define el **espacio de parámetros** (las cuatro dimensiones libres: *activation*, *optimizer*, *lr*, *init*) y genera la grilla completa como **producto cartesiano**: 3 × 2 × 3 × 2 = **36 configuraciones**. Ninguna se escribe a mano. Además expone `validate_config()` , que clasifica cada configuración en **VALID**, **WARNING** o **INVALID** con una justificación explícita.

Qué aporta. Es el **generador y el portero** del experimento. Su sistema de reglas distingue dos cosas: errores **INVALID** (restricciones técnicas reales: un nombre de activación desconocido, dimensiones incompatibles, proporciones que no suman 1, `lr` ≤ 0) que **bloquean** la ejecución; y advertencias **WARNING** (riesgo metodológico: `lr` alto con Adam o SGD, He con sigmoide/tanh, Xavier con ReLU) que **se permiten ejecutar** pero se registran. Por defecto, las 36 combinaciones quedan como máximo en WARNING: **nada se rechaza por mera convención**.

Con quién se relaciona. Toma el esquema de `config.py` y alimenta tanto al `runner.py` (qué entrenar) como al `store.py` (qué registrar, con su estado de validación). También define `compute_space_key()` , la huella de las 4 dimensiones que usa `identity.py` .

4.3 `identity.py` — las "cédulas" de configuraciones y corridas núcleo

Qué hace. Genera **identificadores estables y únicos**. Maneja dos tipos de huella y es clave entender por qué son dos:

- `config_id` = `cfg-v1-<sha256(config completa)[:12]>` . Hashea **todos** los campos, incluida la semilla y el `class_weight` . Dos configuraciones idénticas dan el mismo id; cualquier diferencia da otro. Sirve para **agrupar repeticiones**.
- `space_key` = `space-v1-<hash de las 4 dimensiones>` . Solo depende de activation/optimizer/lr/init, así que un cambio de `class_weight` **no parte** la grilla de 36.

Qué aporta. Resuelve el dilema de tener una identidad "fina" (todo el protocolo) y una "gruesa" (solo el espacio). Además crea los **run_id** como `<timestamp UTC>-<6 hex>` . Ese id es el **nombre de la carpeta** en la que se guarda la corrida, y como las carpetas no se pueden crear dos veces, esto convierte el "nunca sobreescribir" en algo **estructural**, no en una promesa.

Con quién se relaciona. Lo usan `store.py` (nombres de archivos y deduplicación), `runner.py` (id de cada corrida) y `compare.py` (agrupar y seleccionar baseline).

4.4 `store.py` — la memoria del laboratorio núcleo

Qué hace. Maneja la **persistencia en disco** mediante la clase `ResultsStore`. Escribe y lee la estructura de carpetas de cada experimento y ofrece consultas históricas (`query_runs`) que **nunca** dependen de lo que haya en memoria del proceso: siempre leen del disco.

Qué aporta. Es el guardián de la **inmutabilidad**. Cada corrida se crea con operaciones atómicas: primero se crea la carpeta (que falla si ya existe) y luego se escriben `metadata.json` y `epochs.csv` **una sola vez** (con archivo temporal + reemplazo atómico). Los dos logs, `configs.jsonl` y `decisions.jsonl`, son **append-only**: nunca se reescriben. Esto significa que un resultado guardado **no se puede modificar**, solo agregar cosas nuevas.

Con quién se relaciona. Es el punto de contacto con el disco para `runner.py` (guardar), `compare.py` y `aggregate.py` (leer para comparar/agrupar), `decisions.py` (loguear decisiones) y `export.py` (leer todo para exportar).

4.5 `data.py` — los datos: descarga, huella, partición y lotes pipeline

Qué hace. Se ocupa de todo lo relativo a los datos: `fetch_wine_quality_csv()` descarga el dataset *Wine Quality (white)* de UCI y lo cachea localmente; `load_wine_quality()` lo carga; `build_splits()` arma la partición train/val/test 60/20/20 estratificada y sembrada; y `build_dataloaders()` escala con `StandardScaler` y construye los `DataLoader` de PyTorch.

Qué aporta. Dos cosas valiosas. Primero, la **huella del dataset** (`compute_file_fingerprint` / `compute_dataframe_fingerprint`): un `sha256-<digest>` del contenido, para poder **probar** que dos corridas usaron exactamente los mismos datos. Segundo, un escalado correcto: el `StandardScaler` se ajusta **solo con el train** y se aplica a val/test, evitando *fuga de información*. Los lotes usan batch 32 y un generador sembrado para que el mezclado sea reproducible.

Con quién se relaciona. Lo usa casi exclusivamente `runner.py`. Toma los parámetros (split, batch, semilla) de `config.py` y entrega loaders listos para entrenar.

4.6 `model.py` — la red y cómo se inicializan sus pesos pipeline

Qué hace. `build_model()` construye el perceptrón multicapa (MLP) como `nn.Sequential`, insertando la activación configurada en cada capa oculta y dejando la capa de salida lineal. `initialize_weights()` inicializa los pesos con **He/Kaiming** o **Xavier/Glorot** según la configuración, y pone los *biases* en cero.

Qué aporta. Materializa la dimensión "arquitectura" + "activación" + "inicialización" del experimento. Aporta además `parameters_are_finite()`, una comprobación de que ningún peso se volvió *NaN*/*Inf*, usada como red de seguridad durante el entrenamiento.

Con quién se relaciona. Lo usa `runner.py`, tomando la `ArchitectureSpec` y los nombres de activación/inicialización de `config.py`.

4.7 `metrics.py` — medir y cuidar la estabilidad numérica pipeline

Qué hace. Calcula las métricas. `compute_final_metrics()` devuelve exactitud, exactitud balanceada, precisión/recall/F1 macro y la matriz de confusión. `epoch_train_step()` ejecuta una época de entrenamiento; `evaluate()` evalúa el modelo sin gradientes.

Qué aporta. Además de las métricas, aporta una **defensa numérica** importante: ambas funciones detectan valores no finitos (*NaN*/*Inf*) y, en el paso de entrenamiento, **saltan el paso del optimizador** si la pérdida no es finita, para que un gradiente corrupto no envenene los pesos. Devuelven banderas (`non_finite`, `non_finite_batch`) con la época y el lote exactos donde falló.

Con quién se relaciona. Lo usa `runner.py` en cada época y en la evaluación final.

4.8 `runner.py` — el director de orquesta núcleo

Qué hace. `run_experiment()` es el corazón ejecutable. Coordina todo: siembra las fuentes aleatorias (NumPy, torch, CUDA) para reproducibilidad; elige el dispositivo (CPU/CUDA); genera `run_id` y `config_id`; valida y registra la configuración; resuelve el criterio de convergencia; obtiene la huella del dataset y captura el entorno (ambas de forma "best-effort"); y ejecuta el **bucle de entrenamiento** época por época.

Qué aporta. Es donde viven las garantías del experimento:

- **Entrenamiento configurable:** arma dataloaders, modelo, inicialización, optimizador (SGD/Adam) y pérdida (CrossEntropy, con pesos balanceados si se pidió).
- **Parada por convergencia:** si está habilitada, aplica el criterio (por defecto `val_loss`, paciencia 20, min_delta 1e-4) y registra **por qué** paró.
- **Robustez numérica:** si detecta *NaN*/*Inf* en train, validación o en los propios pesos, **aborta** la corrida como FAILED, pero **igual la guarda** con el *traceback* y la época/lote del fallo.
- **Metadatos completos:** arma el registro de la corrida con snapshot de la config, versión del dataset, entorno, nota de determinismo, estado de ejecución, advertencias, métricas finales (etiquetadas como evaluadas en *test*), cantidad de parámetros y tiempo de entrenamiento.
- **Épocas inmutables:** acumula el historial de épocas en memoria y lo escribe **una sola vez** al finalizar (o al fallar), garantizando que no se corrompa a mitad de camino.

Con quién se relaciona. Es el integrador central: usa `data`, `model`, `metrics`, `environment`, `config`, `identity`, `space` y `store`. Todo lo demás existe para servirlo a él o para leer lo que él produce.

4.9 `environment.py` — dejar constancia de dónde se corrió soporte

Qué hace. Captura el **entorno de ejecución**: sistema operativo, CPU, disponibilidad de CUDA y nombre de las GPUs, y las versiones de Python, torch, numpy, pandas, scikit-learn, scipy y matplotlib.

Qué aporta. Reproducibilidad. Saber con qué se corrió permite razonar sobre un resultado más tarde. Cada sonda está envuelta defensivamente: si algo no está disponible, devuelve `None` y **nunca** hace fallar la corrida.

Con quién se relaciona. Lo invoca `runner.py` y su salida se guarda en los metadatos de cada corrida.

4.10 `compare.py` — comparar contra una línea base, con honestidad análisis

Qué hace. Compara corridas contra una baseline. Es un módulo de **funciones puras**: no hace entrada/salida ni usa torch, solo opera sobre los metadatos de las corridas. `select_baseline()` elige la línea base **de forma explícita** (por id de corrida, por id de config, o activando el default documentado): **nunca adivina**. `check_comparability()` verifica que dos corridas sean realmente comparables. `compare_to_baseline()` calcula las diferencias.

Qué aporta. Es la **conciencia metodológica** del proyecto. Antes de emitir un número, comprueba que dataset (nombre + versión), arquitectura, partición, épocas, convergencia, tipo de corrida y conjunto de evaluación coincidan. Si algo no coincide, **niega las diferencias** y marca la fila como `non_comparable` en lugar de inventar un valor. Distingue comparaciones **controladas** (cambia exactamente una de las 4 dimensiones) de **multi-variable** (cambian varias, y entonces lo marca como "solo informativo"). Y deliberadamente **no** calcula un score compuesto: preserva el perfil completo de métricas con sus *trade-offs*.

Con quién se relaciona. Lee los metadatos persistidos por `store.py` (que produce `runner.py`) y expone una **superficie de normalización** (`run_state`, `run_config`, `run_field` …) que reutilizan `aggregate.py` y `export.py` para leer los registros **de la misma forma**.

4.11 `aggregate.py` — medir la variabilidad de las repeticiones análisis

Qué hace. Toma un conjunto de corridas, las **agrupa por identidad de configuración** y calcula estadísticas por grupo. Por defecto agrupa por configuración **normalizada por semilla** (le quita la semilla), para que varias corridas del mismo protocolo con distinta semilla caigan en un mismo grupo. También ofrece el modo estricto `grouping="config_id"`.

Qué aporta. Responde a la pregunta "¿qué tan estable es esta configuración?". Por cada métrica calcula `n`, media, desvío estándar muestral (`ddof=1`), mínimo, máximo y rango. Con honestidad estadística: reporta `n_total`, `n_valid`, `n_failed` y `n_unstable` con sus proporciones; las corridas fallidas se excluyen de las estadísticas pero **nunca se esconden**; y si hay menos repeticiones que el mínimo (por defecto **3**), marca el grupo como **"repeticiones insuficientes"** y sus valores no se presentan como confiables.

Con quién se relaciona. Reutiliza la superficie de normalización de `compare.py`, toma los vocabularios de `decisions.py` y alimenta la tabla de agregados de `export.py`.

4.12 `decisions.py` — el registro de decisiones, manual y con justificación gobernanza

Qué hace. Administra un **log de decisiones manual** (append-only). `record_decision()` agrega una entrada que exige una **justificación no vacía**, un **actor** y una referencia a la entidad afectada (corrida, config o grupo).

Qué aporta. Mantiene **dos clasificaciones independientes**: el **estado técnico de ejecución** (COMPLETED / COMPLETED_WITH_WARNINGS / UNSTABLE / FAILED / INCOMPLETE), que escribe el runner y el log nunca reescribe; y la **etiqueta exploratoria** (VIABLE / INTERESTING / UNSTABLE / NON_VIABLE / PENDING_REVIEW), que decide una persona. Si alguien intenta usar un estado técnico como etiqueta exploratoria, el sistema lo **rechaza**: así se garantiza que lo que "pasó" (técnico) nunca se confunda con lo que "opinamos" (exploratorio). No hay ningún motor automático de clasificación; la decisión siempre es humana y queda registrada.

Con quién se relaciona. Escribe a través de `store.py` y sus vocabularios los consumen `aggregate.py` y `export.py`.

4.13 `export.py` — las tablas finales para analizar salida

Qué hace. `export_all()` produce un conjunto de **CSVs con marca de tiempo** en `results/exports/<experimento>/`: un `master` (todas las corridas), `baseline_comparison` (diferencias vs baseline), `aggregates` (estadísticas por grupo), `epochs` (historial por época), `configs` (definiciones + validación), `decisions` y `warnings_summary`.

Qué aporta. Es una capa de **composición**: no reimplementa nada, junta lo que produjeron `store`, `compare`, `aggregate` y `decisions`. Su regla de oro es la **honestidad de los datos faltantes**: todo valor ausente se escribe como el marcador literal `N/A`, **nunca** como un `0` inventado. Los gráficos opcionales son *no bloqueantes*: si fallan, se registra una advertencia y los CSV se generan igual.

Con quién se relaciona. Es la etapa final del pipeline; lee todo desde `store.py` y reutiliza `compare.py`, `aggregate.py` y `decisions.py`.

4.14 `cli.py`, `__init__.py` y `__main__.py` — la puerta de entrada interfaz

Qué hacen. `cli.py` es un punto de entrada **delgado** por línea de comandos (`python -m prebenchmark`) con cinco subcomandos que mapean 1:1 con las etapas del pipeline: `generate`, `run`, `compare`, `aggregate` y `export`. `__main__.py` habilita esa invocación, y `__init__.py` expone la API pública del paquete. El subcomando `run` acepta `--multi-seed` (corre cada configuración con el conjunto fijo de 5 semillas), `--earlyStop` (enciende el criterio de corte por convergencia) y `--class-weight` (`none` / `balanced`: enciende o apaga el balanceo de clases **sin tocar el espacio de 36 combinaciones**, porque `class_weight` vive fuera de la `space_key`).

Qué aportan. El CLI **no reimplementa** lógica: solo cablea los módulos. Un detalle de diseño fino: importa torch **de forma perezosa** solo dentro del subcomando `run`, de modo que `generate`, `compare`, `aggregate` y `export` funcionan sin cargar PyTorch ni tocar la red. `__init__.py` es **import-safe**: importarlo no dispara entrenamiento ni entrada/salida.

Con quién se relaciona. Es la fachada de todo el paquete hacia quien lo usa desde una terminal.

5. Cómo se relacionan los módulos (mapa mental)

Visto como capas, el sistema queda así:

Capa	Módulos	Rol
Definición	`config.py`, `space.py`, `identity.py`	Definen el *qué*: esquema, grilla de 36 configuraciones, validación e identidades.
Ejecución	`data.py`, `model.py`, `metrics.py`, `runner.py`, `environment.py`	Hacen el *trabajo*: cargar datos, armar y entrenar la red, medir, y registrar el entorno.
Persistencia	`store.py`	Garantiza que nada se pierda y nada se sobreescriba.
Análisis	`compare.py`, `aggregate.py`, `decisions.py`	Interpretan los resultados: comparar, medir variabilidad, registrar decisiones.
Salida	`export.py`, `cli.py`	Publican los resultados (CSV) y ofrecen la interfaz de uso.

El flujo de dependencias es en un solo sentido: las capas de definición no conocen a las de ejecución, y el análisis solo mira lo que quedó persistido en disco. Eso hace que cada pieza sea testeable por separado: las capas puras (`config`, `space`, `compare`, `aggregate`, `decisions`) se prueban sin entrenar, y solo la capa de ejecución necesita PyTorch.

6. Las ideas que sostienen todo el diseño

Trazabilidad total. Cada resultado guarda el snapshot completo de su configuración, la versión exacta del dataset y el entorno. Se puede reconstruir de dónde salió cada número.

Inmutabilidad estructural. Las corridas se escriben una sola vez y su carpeta no puede reutilizarse; los logs son append-only. No hay API para modificar un resultado pasado.

Explicitud por encima de los valores ocultos. Todo valor por defecto (convergencia, baseline, mínimo de repeticiones) está declarado, es configurable y queda registrado. Nada es un umbral mágico.

Honestidad metodológica. Si dos corridas no son comparables, se dice y no se emite el número; si faltan datos, se escribe `N/A` y no un cero; si falla una corrida, se guarda y se reporta el fallo en vez de esconderlo.

Determinismo y reproducibilidad. Semillas fijas para NumPy, torch y CUDA; generadores sembrados en los dataloaders; y una nota que aclara los límites (por ejemplo, que ciertos *kernels* de cuDNN pueden seguir siendo no deterministas).

7. Decisiones arquitectónicas y consideraciones generales

Además de las decisiones de *código*, el proyecto tomó un conjunto de **decisiones metodológicas**: cómo comparar configuraciones de forma justa, cómo medir su estabilidad, qué red usar y con qué técnica de aprendizaje, y cuándo detener el entrenamiento. Este apartado reúne esas definiciones y el razonamiento detrás de cada una.

7.1 Protocolo multi-semilla: cómo medimos la estabilidad

Una configuración que "gana" por un margen chico puede haber ganado por casualidad de la semilla inicial. Para separar señal de ruido, la etapa exploratoria corre cada configuración con un **conjunto fijo de 5 semillas**:

```
EXPLORATORY_SEEDS = (42, 123, 7, 2025, 999)   # prebenchmark/seeds.py
```

- **Cantidad (5).** Es un compromiso razonable: alcanza para estimar la dispersión sin multiplicar el costo por 10 o 20. Las semillas son **fijas** (no aleatorias) para que el barrido sea reproducible y comparable entre corridas.
- **Cómo se comparan.** Cada configuración (protocolo) se agrupa *ignorando* la semilla y se resume con **media ± desvío estándar muestral** (`ddof=1`) sobre `val_loss` y `val_accuracy`. Una configuración es **estable** cuando su dispersión entre semillas es chica.
- **Agrupación.** El `config_id` es sensible a la semilla, pero la agrupación por defecto (`grouping="config"`) la normaliza, de modo que las 5 corridas caen en el mismo grupo. El modo estricto `grouping="config_id"` las mantiene separadas.
- **Implementación.** El flag `--multi-seed` del subcomando `run` expande cada configuración del espacio a las 5 semillas. Sin él, `run` ejecuta una configuración con una sola semilla (`--seed`).

Importante: comparar dos *semillas de la misma configuración* **no** es una comparación multi-variable (ver 7.2). La semilla se promedia; no es una de las cuatro dimensiones del espacio.

7.2 Comparación controlada vs. multi-variable ("solo informativo")

No todas las diferencias son igual de interpretables. `compare.py` distingue dos casos:

- **Controlada.** La comparación cambia **exactamente una** de las cuatro dimensiones del espacio (activation, optimizer, lr, init). El delta es **atribuible** a ese único cambio: por ejemplo, "pasar de SGD a Adam mejoró la pérdida".
- **Multi-variable.** Cambian **varias** dimensiones a la vez. El delta igual se calcula, pero se marca como **"solo informativo"**: no se puede atribuir a un solo factor, porque los cambios se mezclan y sus efectos se confunden (no hay control experimental).

La etiqueta "solo informativo" es una forma de **no mentir con los números**: se muestra el dato, pero se aclara que no admite lectura causal. Antes de emitir *cualquier* diferencia, el módulo comprueba además que las dos corridas sean realmente comparables (mismo dataset y versión, arquitectura, partición, épocas, convergencia, tipo de corrida y conjunto de evaluación); si algo no coincide, la fila queda marcada como `non_comparable` y no se emite el número.

7.3 MLP entrenado por retropropagación, no el perceptrón clásico

El **perceptrón clásico** (la regla de aprendizaje de Rosenblatt) es un modelo de **una sola capa** que solo puede aprender fronteras de decisión **lineales**. Su límite es histórico y documentado: no resuelve problemas que no son linealmente separables (el ejemplo canónico es XOR), y no se extiende de forma natural a varias capas apiladas.

Nuestro problema es de **clasificación multiclase no lineal** (la calidad del vino se reparte en varias clases que no se separan con una recta en el espacio de las 11 variables). Por eso usamos un **perceptrón multicapa (MLP)** —capas ocultas con activaciones no lineales— entrenado por **retropropagación (backpropagation)**: el error se propaga hacia atrás por la red y los pesos de *todas* las capas se ajustan con el gradiente de la función de pérdida. Ese mecanismo es lo que permite aprender representaciones no lineales capa por capa, algo que la regla clásica del perceptrón no puede hacer.

7.4 El criterio de corte (early stopping)

La convergencia está **apagada por defecto**; se enciende con el flag `--earlyStop`. Al encenderse, los campos que no especifiquemos se completan con los valores documentados:

Parámetro	Valor	Qué significa
`metric`	`val_loss`	Qué se vigila para decidir el corte.
`patience`	`20`	Cuántas épocas sin mejora se toleran antes de cortar.
`min_delta`	`1e-4`	Cuánto tiene que mejorar para contar como "mejora".
`mode`	`min`	Menor pérdida es mejor.
`restore_best_weights`	`True`	Al cortar, restaura los mejores pesos (ver 7.5).

¿Por qué se mide la pérdida del conjunto de validación y no la del entrenamiento? Porque la pérdida de **entrenamiento casi siempre baja**: el modelo se ajusta a los datos que ve. Si cortáramos por ella, no cortaríamos nunca, porque el modelo seguiría "mejorando" en el train mientras **sobreajusta** y empeora en datos nuevos. La pérdida de **validación** se calcula con datos que el modelo **no** usa para ajustar sus pesos, así que refleja **capacidad de generalización**. Cuando la validación deja de mejorar, seguir entrenando solo agrega sobreajuste. Por eso el corte honesto se basa en `val_loss`.

La paciencia (20) existe porque `val_loss` no baja de forma perfectamente monótona: puede subir un poco por ruido y luego volver a bajar. Exigir 20 épocas sin mejorar evita cortar por una fluctuación pasajera. El `min_delta` (1e-4) evita considerar "mejora" a variaciones insignificantes.

7.5 Guardar los pesos de la mejor época (`restore_best_weights`)

Cuando el criterio está habilitado, se guarda el **estado de los pesos en la época de menor** `val_loss` y, al terminar el bucle, se **restauran** antes de evaluar en test. Es la semántica de `restore_best_weights=True` de Keras.

El motivo: la última época **no** es necesariamente la mejor. Con early stopping el entrenamiento se detiene justo cuando la validación dejó de mejorar, así que la última época puede estar peor que el mejor punto anterior. Si evaluáramos el test con los pesos de la última época, reportaríamos un resultado **peor del que la configuración realmente logró**. Restaurar los mejores pesos hace que las métricas finales correspondan al mejor punto alcanzado. Aplica tanto si la corrida corta por convergencia como si llega al tope de épocas.

8. Resultados de los análisis obtenidos

Antes de lanzar el barrido completo (36 configuraciones × 5 semillas = **180 corridas**), corrimos una serie de **pruebas exploratorias** para dimensionar el experimento y tomar decisiones con datos en la mano. Estos son los hallazgos.

8.1 El early stopping cambió por completo el costo del barrido

Primero medimos el costo de entrenar **sin** criterio de corte. Con el tope de 1000 épocas por corrida, el barrido completo (180.000 épocas en total) se estimaba en **~11,5 horas** en esta máquina, con el proceso corriendo en un solo hilo de CPU.

Después activamos el early stopping sobre una muestra de configuraciones representativas y volvimos a medir. El resultado fue contundente: **la enorme mayoría de las configuraciones corta mucho antes de las 1000 épocas**, así que el barrido completo bajó a un orden de **~1,8 horas** (rango aproximado de 0,6 a 2,5 h según qué tan temprano converge cada familia).

Escenario	Tiempo estimado del barrido completo (180 corridas)
Sin early stopping (tope 1000 épocas)	~11,5 h
Con early stopping	~1,8 h

La conclusión práctica: el early stopping no es un detalle de optimización, es lo que hace **viable** el barrido. Multiplica por ~6 la velocidad efectiva del experimento sin sacrificar la calidad del resultado (al contrario: evita evaluar modelos sobreajustados).

8.2 ¿En cuántas épocas convergen la mayoría de las combinaciones?

Sobre una muestra balanceada de 9 configuraciones con early stopping, las épocas de parada fueron estas:

Configuración (activation / optimizer / lr / init)	Época de parada
relu / adam / 0.01 / he	36
tanh / sgd / 0.1 / he	40
relu / sgd / 0.1 / xavier	41
sigmoid / adam / 0.01 / xavier	47
tanh / adam / 0.001 / xavier	80
sigmoid / sgd / 0.1 / he	154
tanh / sgd / 0.01 / he	191
sigmoid / adam / 0.001 / xavier	323
relu / sgd / 0.001 / xavier	1000 (no cortó)

El resumen: mediana **80 épocas**, mínimo 36, y 8 de 9 configuraciones cortaron solas. Las que "ganan" convergen muy temprano — los mejores resultados aparecen en las épocas **16 a 303**—, mientras que una única familia se fue hasta el tope.

8.3 La familia `lr = 0.001` + `sgd`: un problema de convergencia

La excepción de la sección anterior no fue casual: **toda** la familia que combina una tasa de aprendizaje muy baja (0.001) con SGD resultó patológica en este problema. Estas son las 6 configuraciones de esa familia, todas forzadas a correr hasta el tope:

Configuración (activation / init)	Época de parada	Mejor `val_loss`
relu / xavier	1000	1.0550
relu / he	1000	1.0675
tanh / xavier	988	1.0680
tanh / he	1000	1.0716
sigmoid / xavier	1000	1.1646
sigmoid / he	1000	1.1680

Dos observaciones clave. Primero, **5 de 6 llegaron al tope con la mejor época pegada al final** (`best_epoch ≈ 1000`): no convergieron, sino que la curva todavía estaba bajando y el límite de épocas las **truncó**. Segundo, ninguna se acerca a las mejores configuraciones: una configuración rápida como `relu / sgd / 0.1 / xavier` alcanza `val_loss ≈ 1.029` en **21 épocas**, mejor que lo que esta familia logra en 1000. Es decir, **esta familia no es competitiva**, y su mala convergencia es una propiedad real del espacio de búsqueda, no un artefacto del tope de épocas.

8.4 Por qué bajamos el tope por defecto a 500 épocas

Con esos datos, dejar el tope en 1000 épocas no tenía sentido: los resultados competitivos aparecen entre las épocas 16 y 303, y la única familia que llega a 1000 **no mejora el ranking** aunque se entrene más. Por eso el valor por defecto pasó de **1000 a 500 épocas**.

El tope no es una meta de entrenamiento, es un **presupuesto de terminación**: un límite de seguridad para las configuraciones que no convergen. Bajarlo a 500 recorta tiempo en el peor caso sin afectar a las configuraciones que compiten. El `min_delta` del criterio de corte se dejó intencionalmente sin cambios.

8.5 CPU vs. GPU: por qué la GPU no aportaba mejoras

La máquina tiene una **NVIDIA RTX 4050 Laptop** (6 GB), pero el entorno de PyTorch instalado es **CPU-only** (`torch 2.14.1+cpu` , `cuda.is_available() = False`), así que la GPU no era usable sin reinstalar. Más allá de eso, **aunque estuviera disponible, no ayudaría**:

- El modelo es **minúsculo**: un MLP de unos **1031 parámetros** (11 → 32 → 16 → 7).
- Los lotes son chicos (batch 32) y el dataset también, así que una época son apenas ~153 lotes.
- Cada paso es tan chico que el tiempo no lo domina el cálculo (FLOPs) sino el **overhead de despacho** (lanzar operaciones, mover datos).

En un caso así, mover el cómputo a la GPU agrega **lanzamiento de kernels y transferencias host↔device** que ni siquiera se amortizan: para redes tan chicas y lotes tan chicos, la GPU suele *empatar* o incluso *ir más lento* que la CPU. La conclusión fue **descartar la GPU** y optimizar donde sí había margen: reduciendo el costo por corrida (early stopping y tope de épocas).

8.6 Resumen de tiempos

Medición	Valor
Costo por época (CPU, esta máquina)	~0,067 – 0,23 s (varía por sesión)
Overhead por corrida	~0,5 s
Arranque del proceso	~15 – 20 s
Barrido completo sin early stopping (tope 1000)	~11,5 h
Barrido completo con early stopping	~1,8 h
Época mediana de convergencia (muestra)	80 épocas
Época de mejor resultado (configuraciones ganadoras)	16 – 303

Todas las corridas del barrido se ejecutan en **un solo proceso** (con `--limit 36 --multi-seed`), de manera que el arranque se paga una sola vez y no una vez por configuración.

8.7 Barrido con/sin balanceo de clases (dos brazos) sobre wine_quality

El dataset `wine_quality` (UCI winequality-white, 4898×12) presenta un desbalance extremo entre 7 clases (quality 3–9): proporciones desde 0,10% (clase 9) hasta 44,88% (clase 6), con ratio máximo ~440:1. Antes del barrido completo de 36 configuraciones × 5 semillas se verificó que el código **no aplicaba** `class_weight` (queda en `None` por construcción) y que existía una fuga: el `criterion` ponderado se reutilizaba en validación/test. Se corrigió para separar `criterion` de entrenamiento (ponderado con pesos por frecuencia sobre `y_train`) y evaluación/early stopping (siempre sin ponderar), se añadió `--class-weight {none,balanced}` a `generate/run` y `compare` pasó a tratar `class_weight` como dimensión controlada.

Se ejecutó el experimento `exp-sweep-01` con early stopping (`metric=val_loss, patience=20, min_delta=1e-4`) en **dos brazos secuenciales**: 36 configs × 5 semillas = **180 corridas** sin balanceo (`class_weight=None`) y **180 corridas** con balanceo por frecuencia (`class_weight=balanced`), total **360 corridas** completadas en ~1h07m.

Arm	accuracy (media ± std)	balanced_accuracy (media ± std)	precision_macro	recall_macro	f1_macro	loss (sin ponderar)	Épocas (media/mediana/max)	Tiempo total
Sin balanceo (`none`)	0.5470 ± 0.0272	0.2396 ± 0.0256	0.2890 ± 0.0636	0.2396 ± 0.0256	0.2402 ± 0.0360	1.0719 ± 0.0517	182.4 / 70 / 500	39.3 min
Con balanceo	0.4245 ± 0.0487	0.3034 ± 0.0627	0.2484 ± 0.0581	0.3034 ± 0.0627	0.2486 ± 0.0575	1.3331 ± 0.1145	123.0 / 68 / 500	27.2 min

Arm	accuracy (media ± std)	balanced_accuracy (media ± std)	precision_macro	recall_macro	f1_macro	loss (sin ponderar)	Épocas (media/mediana/max)	Tiempo total
(balanced)								
Diferencia (balanced - none)	**-0.1225**	**+0.0638**	**-0.0406**	**+0.0638**	**+0.0084**	**+0.2612**	-59.4	-12.1 min

Comparación pareada por (configuración del espacio, semilla): **balanced_accuracy** gana en **166/180** pares (mediana +0.0666), **f1_macro** en **119/180** (mediana +0.0147). El **accuracy** cae sistemáticamente (-0.1225 medio) porque el clasificador deja de favorecer mayoritariamente la clase 6. El `loss` reportado en test/validación es siempre **sin ponderar** (tras la corrección) para mantener comparabilidad entre brazos.

Rank (bal_acc)	Configuración	Arm	balanced_accuracy	accuracy	f1_macro	loss	épocas
1	(tanh, adam, 0.001, xavier)	balanced	0.3666 ± 0.0567	0.4351	0.2830	1.2584	116
2	(tanh, adam, 0.01, he)	balanced	0.3609 ± 0.0422	0.4610	0.3034	1.2373	46
3	(tanh, sgd, 0.1, he)	balanced	0.3557 ± 0.0451	0.4571	0.2953	1.2412	50
4	(relu, adam, 0.001, he)	balanced	0.3508 ± 0.0108	0.4499	0.2840	1.2696	216
5	(relu, adam, 0.01, he)	balanced	0.3504 ± 0.0938	0.4372	0.2849	1.3042	148
Mejor sin balanceo	(tanh, adam, 0.001, xavier)	none	0.2606 ± 0.0136	0.5584	0.2714	1.0350	78

La agregación por `config_id` del espacio generó **72 grupos** (36 configuraciones × 2 brazos) con 5 semillas válidas cada uno. Las comparaciones cruzadas entre brazos fueron marcadas como **controladas** por `class_weight` por `prebenchmark/compare.py`. Los artefactos exportados quedaron en `results/exports/exp-sweep-01/` (`master` , `baseline_comparison` , `aggregates` , `epochs` , `configs` , `decisions` , `warnings_summary`), incluyendo comparaciones controladas específicas de `class_weight` .

Documento generado a partir de la lectura directa del código de los 16 módulos del paquete `prebenchmark/` y del diseño en `openspec/changes/prebenchmark-infra/design.md` . Refleja el estado actual del repositorio.

9. Cómo registramos los resultados y cómo hacemos el análisis

El diseño es intencionalmente simple: una capa **primaria** (lo que escribe el `runner`) y una capa **derivada** (lo que genera `export.py` para analizar rápido).

9.1 ¿Dónde queda todo?

- `results/<experiment_id>/` - todo lo generado para ese experimento.
- `results/<experiment_id>/configs.jsonl` - definiciones de configuraciones ejecutadas para ese experimento (una línea por `config_id` /instancia). Incluye metadatos de validación (`validation_state` , `validation_rationale`) y `space_key` para agrupar por espacio.
- `results/<experiment_id>/<run_id>/` - una carpeta por corrida. Aquí está lo mínimo para reproducir/auditar esa corrida exacta.
- `results/exports/<experiment_id>/` - artefactos **derivados** en CSV, con marca de tiempo UTC (no se sobrescriben entre exportaciones).

9.2 Dentro de cada carpeta de corrida (`<run_id>/`)

> - `metadata.json` - **Snapshot completo de la corrida**. Contiene `config_snapshot` (el `ExperimentConfig` tal como se usó, `cfg.to_dict()`), campos denormalizados a nivel raíz (`dataset` , `architecture` , `seed` , `activation` , `optimizer` , `lr` , `init` , `batch_size` , `max_epochs` , `split` , `preprocessing` , `convergence`), estado de ejecución (`execution_state`), contexto (`hardware` , `library_versions` , `determinism`), métricas finales (`final_metrics` en test), `validation_metrics` (mejor época según `val_loss`), `convergence_trigger` (criterio resuelto + razón/parada), parámetros `n_params` , `training_time` , etc. La duplicación es intencional (provenance + acceso rápido para agregación/export).
> - `epochs.csv` - historial por época (`run_id` , `config_id` , `epoch` , `train_loss` / `train_acc` , `val_loss` / `val_acc` , `epoch_time`). Permite ver curvas, justificar el `best_epoch` del `validation_metrics` y auditar `early_stopping` . Con `--`

> `earlyStop` queda mucho más compacto que correr a `max_epochs`.

9.3 Los archivos en `results/exports/<experiment_id>/`

- `configs_<ts>.csv` - Vista aplanada de las configuraciones ejecutadas. Ideal para ver el espacio sin abrir JSON.
- `epochs_<ts>.csv` - Historial plano por época para todo el experimento (long format). **Nota:** puede crecer, pero con early stopping es manejable (herramientas como `polars` o `duckdb` permiten filtrar sin cargar todo).
- `master_<ts>.csv` - Tabla maestra: une config + corrida + métricas finales. **El más práctico** para comparar muchas corridas de un vistazo.
- `aggregates_<ts>.csv` - Agregados por grupo de repeticiones (media/desvío/mín/máx) generados por `aggregate.py`. Sirven para ver estabilidad entre semillas.
- `baseline_comparison_<ts>.csv` - Comparación vs la línea base (generado por `compare.py`). Incluye deltas/diferencias para evaluar mejoras/degradaciones.
- `decisions_<ts>.csv` - Registro de decisiones de validación/gobernanza (proveniente de `decisions.py`), con justificación.
- `warnings_summary_<ts>.csv` - Resumen rápido de `warnings`, fallos o ejecuciones a revisar.

9.4 Cómo leerlo sin cargar todo en memoria

- **Para entender una corrida puntual:** mirar `metadata.json` (snapshot + métricas) + `epochs.csv` de esa carpeta.
- **Para comparar muchas corridas:** usar `master_*.csv`.
- **Para analizar curvas/convergencia:** filtrar `epochs_*.csv` por `run_id` / `config_id` (o usar `duckdb/polars` con filtros).
- **Para estabilidad entre semillas:** mirar `aggregates_*.csv`.

Nota: los CSV de `exports/` son derivados. La fuente primaria sigue siendo `results/<exp>/<run_id>/{metadata.json,epochs.*}` y `configs.jsonl`.

10. Cómo interaccionar con el sistema (CLI)

Toda la infraestructura se maneja desde una única puerta de entrada: `python -m prebenchmark`. La CLI es **delgada**: no reimplementa entrenamiento, persistencia ni análisis; solo valida argumentos y cablea los módulos que ya existen. Sus cinco subcomandos siguen el mismo orden del pipeline:

```
generate ──► run ──► compare ──► aggregate ──► export
(registrar)   (ejecutar)  (comparar)   (agrupar)      (publicar)
```

Cada subcomando escribe en un **experimento** identificado por `--experiment-id`. El experimento es, literalmente, el nombre de la carpeta dentro de la raíz de resultados (`results/` por defecto). Por eso conviene usar identificadores estables, cortos y sin barras ni caracteres reservados del sistema de archivos.

> **Regla de oro.** Los comandos **no comparten estado en memoria**: cada uno vuelve a leer del disco lo que dejó el anterior. Se pueden ejecutar en sesiones distintas, en otro orden parcial o repetir sin miedo, porque las corridas y los logs son inmutables o *append-only*. Lo único que cambia es qué habrá disponible para leer.

10.1 Punto de entrada, opciones comunes y códigos de salida

Se ejecuta desde la raíz del repositorio, con el entorno virtual activado (o llamado directamente):

```
python -m prebenchmark --help
python -m prebenchmark <subcomando> --help
```

En Windows/PowerShell, si el entorno no está activado:

```
.\.venv\Scripts\Activate.ps1
python -m prebenchmark generate --experiment-id exp-001
```

```
# Alternativa sin activar el entorno:
& .\.venv\Scripts\python.exe -m prebenchmark generate --experiment-id exp-001
```

La forma general de cualquier comando es:

```
python -m prebenchmark {generate,run,compare,aggregate,export} --experiment-id EXP [--root results]
```

Opción común	Obligatoria	Valor por defecto	Qué hace
`--experiment-id`	Sí, en todos	—	Selecciona (o crea) el experimento. Es el nombre de la carpeta `results/<EXP>/`.
`--root`	No	`results`	Raíz del almacén de resultados. Cambiarla permite tener laboratorios separados (por ejemplo, `--root pruebas`) sin mezclar corridas.
`--help` / `-h`	No	—	Muestra la ayuda del comando y termina.

Los comandos de análisis (`compare`, `aggregate` y `export`) comparten además tres opciones de línea base. Su intención es que la línea base **nunca se adivine**: hay que elegirla explícitamente.

Opción de línea base	Efecto
`--baseline-run-id <RUN_ID>`	Usa exactamente esa corrida como línea base. Debe existir y no estar `FAILED`.
`--baseline-config-id <CONFIG_ID>`	Busca corridas con ese `config_id` y elige la más temprana (por `start_time`). Esa corrida no debe estar `FAILED`.
`--documented-default`	Opta por la línea base documentada del repositorio: **ReLU + Adam + lr 0.001 + He + CrossEntropy + wine_quality**. Solo funciona si existe una corrida con *exactamente* ese protocolo (semilla 42, 500 épocas, sin early stopping y sin `class_weight`).

Se pasa **como máximo una** de las tres. `compare` la exige; `export` la usa si se indica y, si no, reutiliza la que quedó guardada en `experiment.json`; `aggregate` acepta las opciones por compartir el parser, pero **no las usa**.

Códigos de salida. La CLI distingue tres situaciones:

Código	Cuándo ocurre	Qué significa
`0`	El comando terminó su trabajo.	Éxito del *comando*. Ojo: en `run`, una corrida puede quedar `FAILED` y el comando igualmente devuelve `0`.
`1`	Error controlado: `ValueError` o `FileNotFoundError`.	Se imprime `error: <motivo>` en **stderr**. Por ejemplo, una línea base inexistente o un `--config-id` que no pertenece a la grilla.
`2`	No se indicó subcomando, o los argumentos son inválidos.	Se muestra la ayuda (o el error de `argparse`) y no se ejecuta nada.

Los mensajes de consola están en inglés porque provienen directamente de `argparse` y de los `print` del código. La documentación y los artefactos son independientes de eso.

Lo que la CLI no tiene. No hay un subcomando para listar corridas ni para registrar decisiones: los `run_id` se obtienen de la salida de `run` o de las carpetas `results/<EXP>/`, y las decisiones se registran por API (`ResultsStore.append_decision`) o como efecto de `compare` / `export`. Tampoco hay un flag para forzar `device`; eso vive en la configuración del experimento.

10.2 `generate`: registrar la grilla de candidatos

Para qué sirve. Genera el producto cartesiano de las cuatro dimensiones libres (`activation` × `optimizer` × `lr` × `init`), valida cada combinación y la registra en el experimento. Es la etapa de **pre-registro**: deja constancia de qué se va a probar y con qué estado de validación, antes de gastar tiempo de cómputo.

```
python -m prebenchmark generate --experiment-id exp-001
python -m prebenchmark generate --experiment-id exp-001 --limit 6
python -m prebenchmark generate --experiment-id exp-001 --class-weight balanced
```

Opción	Valor	Qué hace
`--limit`	entero, opcional	Registra solo las primeras *N* combinaciones de la grilla, en el orden del producto cartesiano. Sin `--limit` se registran las 36.
`--class-weight`	`none` o `balanced`	Sustituye `preprocessing.class_weight` en lo que se registra. Omitirlo deja el valor tal como lo genera el espacio (sin balanceo). `none` lo desactiva explícitamente; `balanced` lo activa.

Qué hace exactamente. Recorre la grilla completa en el orden `activation → optimizer → lr → init`. Por eso las primeras combinaciones son `relu/sgd/0.001/xavier`, `relu/sgd/0.001/he`, `relu/sgd/0.01/xavier`, etc. Para cada una aplica el override de `class_weight`, ejecuta `validate_config` y llama a `ResultsStore.register_config`. El registro es **append-only y deduplicado por `config_id`**: volver a registrar la misma configuración no la duplica ni la reescribe.

Salida esperada. Una línea con el total registrado y el conteo por estado de validación:

```
registered 36 configuration(s) into 'exp-001': {'WARNING': 24, 'VALID': 12}
```

Los números son un ejemplo de la grilla completa: por defecto, 24 combinaciones quedan en `WARNING` (ejecutables, con un riesgo metodológico registrado) y 12 en `VALID`. Con `--limit`, el total baja y los conteos cambian. Con `--class-weight`, los estados no cambian, pero sí cambia el `config_id` de cada configuración.

Qué deja en disco. Solo `results/<EXP>/configs.jsonl`. No crea carpetas de corrida ni entrena nada; por eso es seguro ejecutarlo sin PyTorch cargado.

- **El registro no es la ejecución.** `run` vuelve a generar la grilla por su cuenta; no lee `configs.jsonl`. `generate` sirve para dejar la intención registrada y para inspeccionar la validación, no como requisito técnico para entrenar.
- **Si vas a ejecutar un brazo con balanceo**, repetí `--class-weight balanced` también en `run`. El `config_id` cambia con el balanceo, pero la `space_key` no: el espacio de 36 sigue siendo el mismo.
- **No acepta** `--max-epochs`, `--seed` ni `--earlyStop`. Las variantes de protocolo se registran recién cuando se ejecutan, con su propio `config_id`.

10.3 `run`: ejecutar configuraciones

Para qué sirve. Entrena una o más configuraciones de la grilla y persiste cada corrida con su snapshot, su historial de épocas y sus métricas. Internamente regenera el espacio con `space.generate_configs()`, aplica los overrides de la CLI y llama a `runner.run_experiment`.

```
python -m prebenchmark run --experiment-id exp-001
python -m prebenchmark run --experiment-id exp-001 --limit 36 --max-epochs 200 --earlyStop
python -m prebenchmark run --experiment-id exp-001 --limit 36 --multi-seed
python -m prebenchmark run --experiment-id exp-001 --limit 1 --class-weight balanced
python -m prebenchmark run --experiment-id exp-001 --config-id cfg-v1-118298388a87
```

Opción	Valor por defecto	Qué hace
`--limit`	`1`	Cantidad de **configuraciones** a ejecutar, no de corridas. El total de corridas es `limit × semillas`. El valor por defecto es deliberadamente conservador: una configuración si no se indica nada.
`--config-id`	—	Ejecuta solo la configuración de la grilla generada cuyo `config_id` coincida. Se aplica **antes** de los overrides de semilla, épocas y convergencia (ver limitaciones).
`--max-epochs`	`500`	Override del tope de épocas. Es un presupuesto de terminación, no una garantía de convergencia.
`--seed`	`42`	Override de la semilla aleatoria para una corrida única.
`--multi-seed`	desactivado	Ejecuta cada configuración una vez por cada semilla del conjunto fijo `(42, 123, 7, 2025, 999)`. Si se usa junto con `--seed`, gana `--multi-seed`.
`--earlyStop`	desactivado	Enciende el criterio de convergencia. Los campos no especificados se resuelven con los defaults documentados: `val_loss`, paciencia 20, `min_delta` `1e-4`, modo `min`, `restore_best_weights=True`. El nombre lleva mayúscula en la **S**.

Opción	Valor por defecto	Qué hace
`--class-weight`	sin cambio	`none` desactiva el balanceo; `balanced` lo activa. El balanceo se aplica **solo al entrenamiento**; validación y prueba se evalúan siempre sin ponderar.

Salida esperada. Una línea por corrida y un resumen final:

```
run 20261022T143012Z-a1b2c3 config cfg-v1-118298388a87 seed 42 state COMPLETED epochs 187
executed 1 run(s) for 'exp-001'
```

La forma exacta de cada línea es: `run <run_id> config <config_id> seed <semilla> state <estado> epochs <épocas ejecutadas>`. Con `--multi-seed` se imprime una línea por cada semilla y el resumen final cuenta todas las corridas. Los estados que produce el runner son `COMPLETED`, `COMPLETED_WITH_WARNINGS` y `FAILED`.

Qué deja en disco. Por cada corrida, una carpeta `results/<EXP>/<run_id>/` con `metadata.json` y `epochs.csv`; además, la configuración ejecutada se registra en `configs.jsonl` (con deduplicación por `config_id`). El `metadata.json` incluye el snapshot completo, la versión del dataset, el entorno, el estado de ejecución, las métricas finales sobre `test`, las métricas de la mejor época sobre `validation` y el disparador de convergencia.

- **Límite de `--config-id`.** El filtro se aplica sobre la grilla recién generada, con sus valores por defecto (semilla 42, 500 épocas, sin convergencia, sin `class_weight`). No busca en las corridas ya persistidas. Por eso un `config_id` obtenido de una corrida con `--seed`, `--max-epochs`, `--earlyStop` o `--class-weight` **no coincide** y el comando falla con `no generated configuration has config_id ...`. En esos casos, usá `--limit` o la API programática.
- **Un fallo no detiene el lote.** El runner captura la excepción, persiste la corrida como `FAILED` con el motivo y el traceback, y el bucle continúa con la siguiente configuración. Por eso el código de salida sigue siendo `0`: hay que mirar los estados, no solo el exit code.
- **Costo.** La grilla completa con `--multi-seed` son 36 × 5 = **180 corridas**, cada una con hasta 500 épocas si no se usa `--earlyStop`. Para una primera prueba conviene `--limit` bajo y `--max-epochs` bajo.
- **Datos.** La primera corrida descarga *Wine Quality (white)* de UCI y lo cachea en `.cache/prebenchmark/winequality-white.csv`. Después del primer uso, el pipeline puede correr sin red.

10.4 `compare`: comparar contra una línea base

Para qué sirve. Elige una línea base y compara **todas las demás corridas** del experimento contra ella, aplicando primero las guardas de comparabilidad. No calcula un ranking ni una puntuación compuesta: produce una tabla de filas con el tipo de comparación y el veredicto de comparabilidad.

```
python -m prebenchmark compare --experiment-id exp-001 --baseline-run-id 20261022T143012Z-a1b2c3
python -m prebenchmark compare --experiment-id exp-001 --baseline-config-id cfg-v1-118298388a87
python -m prebenchmark compare --experiment-id exp-001 --documented-default
```

Reglas de selección de la línea base.

- Hay que indicar **exactamente una** de las tres opciones. Sin ninguna, `compare` termina con error y código `1`.
- Con `--baseline-run-id`, la corrida debe existir y no estar `FAILED`.
- Con `--baseline-config-id`, se elige la corrida **más temprana** que tenga ese `config_id`. Si esa corrida está `FAILED`, la comparación se rechaza: no se salta automáticamente a la siguiente.
- Con `--documented-default`, se busca el `config_id` exacto de la configuración documentada. Si las corridas usaron `--max-epochs`, `--earlyStop`, otra semilla o `class_weight`, ese id **no va a existir**; en ese caso hay que usar `--baseline-run-id`.

Salida esperada. Primero la línea base elegida con su justificación; después, una línea por candidata:

```
baseline 20261022T143012Z-a1b2c3 (cfg-v1-118298388a87): documented default baseline: ReLU + Adam + lr 0.001 + He +
CrossEntropy + wine_quality architecture (the only configuration previously run in this repository)
  20261022T150233Z-d4e5f6 state=COMPLETED type=controlled comparable=True
  20261022T151044Z-0a1b2c state=COMPLETED type=multi_variable comparable=True
  20261022T152155Z-7c8d9e state=FAILED type=non_comparable comparable=False
```

El campo `type` resume la relación entre candidata y línea base:

`comparison_type`	Significado
`identical`	No difiere ninguna dimensión abierta ni `class_weight`.
`controlled`	Difiere **exactamente una** dimensión (activation, optimizer, lr, init o `class_weight`). Es la comparación atribuible a una sola causa.
`multi_variable`	Difieren dos o más dimensiones. Los deltas siguen mostrándose, pero son **solo informativos**: no se puede atribuir la diferencia a una única variable.
`non_comparable`	Falló alguna guarda: dataset o versión distintos, arquitectura distinta, split distinto, `max_epochs` distinto, convergencia distinta, `run_type` distinto, conjunto de evaluación no etiquetado o corrida `FAILED`. No se emiten deltas.

Qué deja en disco. La selección de línea base se guarda en `results/<EXP>/experiment.json` y se agrega una decisión de validación a `decisions.jsonl`. Ese registro es justamente el que después reutiliza `export` si no se le pasa una línea base explícita. La tabla de deltas en sí **no** se imprime en consola: se materializa recién en `baseline_comparison_<ts>.csv` al exportar. `compare` no escribe ninguna carpeta de corrida.

Las corridas candidatas fallidas se listan con `type=non_comparable` y `comparable=False`, pero nunca aportan diferencias. Una línea base fallida se rechaza directamente.

10.5 `aggregate`: agrupar repeticiones

Para qué sirve. Agrupa las corridas que representan el mismo protocolo y calcula, por grupo, cuántas repeticiones hay, cuántas son válidas y si alcanzan el mínimo configurado. Es la vista de **estabilidad**: sirve para saber si una configuración se corrió suficientes veces y si su dispersión es analizable.

```
python -m prebenchmark aggregate --experiment-id exp-001
python -m prebenchmark aggregate --experiment-id exp-001 --grouping config_id
python -m prebenchmark aggregate --experiment-id exp-001 --grouping config --min-repetitions 5
```

Opción	Valor por defecto	Qué hace
`--grouping`	`config`	`config` agrupa por configuración **normalizada por semilla**: las 5 repeticiones multi-semilla del mismo protocolo caen en un solo grupo. `config_id` es el modo estricto y sensible a la semilla: cada semilla queda en su propio grupo.
`--min-repetitions`	`3`	Mínimo de corridas válidas para considerar suficiente el grupo. Si hay menos, se marca `insufficient=True`, pero los valores siguen mostrándose.

Salida esperada. Una línea por grupo y un total:

```
group grp-v1-9f8e7d6c5b4a config cfg-v1-118298388a87 n_valid=5/5 insufficient=False
group grp-v1-1a2b3c4d5e6f config cfg-v1-f415bb95bbbc n_valid=3/5 insufficient=False
2 repetition group(s) for 'exp-001'
```

La forma es `group <group_key> config <config_id> n_valid=<válidas>/<totales> insufficient=<True|False>`. `n_valid` excluye las corridas `FAILED`; `n_total` las incluye. Las corridas inestables (si fueron etiquetadas como tales) sí contribuyen a las estadísticas, pero se divulgan por separado en el CSV.

Qué deja en disco. Nada: `aggregate` es una vista de consola. Las estadísticas completas (n, media, desvío estándar muestral, mínimo, máximo y rango para cada métrica) se escriben en `aggregates_<ts>.csv` durante `export`. Las métricas agregadas son exactitud, exactitud balanceada, precisión/recall/F1 macro, pérdida, tiempo de entrenamiento y las métricas de validación de la mejor época (`val_loss`, `val_accuracy`).

- Las opciones de línea base se aceptan pero **se ignoran** en este comando.
- `export` no expone `--grouping` ni `--min-repetitions`: siempre exporta con `grouping="config"` y mínimo 3. Si necesitás otra agrupación para el CSV, hay que llamar a la API de `export_all`.

10.6 `export`: generar las tablas CSV

Para qué sirve. Lee todo lo persistido del experimento y escribe las tablas derivadas en CSV, con marca de tiempo UTC, para analizarlas con cualquier herramienta. Es la etapa de publicación; no modifica las corridas.

```
python -m prebenchmark export --experiment-id exp-001
python -m prebenchmark export --experiment-id exp-001 --documented-default
python -m prebenchmark export --experiment-id exp-001 --baseline-run-id 20261022T143012Z-a1b2c3
python -m prebenchmark export --experiment-id exp-001 --out .\salidas --no-plots
```

Opción	Valor por defecto	Qué hace
`--out`	`<root>/exports`	Raíz de exportaciones. Los archivos se escriben en `<out>/<EXP>/`.
`--no-plots`	gráficos activados	Omite los PNG opcionales. Los gráficos son de mejor esfuerzo: si fallan, se registra una advertencia y los CSV se generan igual.
Opciones de línea base	ninguna	Si se indica una, se usa y se persiste. Si no, se reutiliza la guardada en `experiment.json`. Si tampoco hay, `baseline_comparison` se exporta sin deltas y se emite una advertencia.

Salida esperada. Una línea con el total y el detalle de cada tabla:

```
exported 7 table(s) to results\exports\exp-001
  master: master_20261022T153500Z.csv (<n> rows)
  baseline_comparison: baseline_comparison_20261022T153500Z.csv (<n> rows)
  aggregates: aggregates_20261022T153500Z.csv (<n> rows)
  epochs: epochs_20261022T153500Z.csv (<n> rows)
  configs: configs_20261022T153500Z.csv (<n> rows)
  decisions: decisions_20261022T153500Z.csv (<n> rows)
  warnings_summary: warnings_20261022T153500Z.csv (<n> rows)
```

Las siete tablas y su contenido son:

Archivo	Contenido
`master_<ts>.csv`	Una fila por corrida (incluidas las `FAILED`): configuración, estado, entorno mínimo, métricas finales de `test` y métricas de validación de la mejor época. Es la tabla más práctica para comparar de un vistazo.
`baseline_comparison_<ts>.csv`	Deltas absolutos y relativos frente a la línea base, más el tipo de comparación, la variable controlada, las advertencias y el veredicto de comparabilidad.
`aggregates_<ts>.csv`	Estadísticas por grupo de repeticiones: n, media, desvío muestral, mínimo, máximo y rango para cada métrica agregada.
`epochs_<ts>.csv`	Historial plano por época de todas las corridas (formato largo), útil para curvas y para auditar el `best_epoch`.
`configs_<ts>.csv`	Vista aplanada de las configuraciones registradas, con estado y justificación de validación, `space_key` y todos los parámetros del protocolo.
`decisions_<ts>.csv`	Registro de decisiones manuales y de validación, con actor, justificación y evidencia.
`warnings_summary_<ts>.csv`	Advertencias por corrida, fallos y decisiones pendientes de revisión (`PENDING_REVIEW`).

- **Exportaciones separadas por marca de tiempo.** Cada exportación lleva un sufijo UTC (`AAAAMMDDThhmmssZ`), así que dos exportaciones en segundos distintos dejan dos juegos de archivos. Si se lanzan dos dentro del mismo segundo, el sufijo coincide; conviene espaciarlas.
- **Los faltantes son `N/A`.** Un delta no comparable, una métrica ausente o una configuración desconocida se escribe como el literal `N/A`, nunca como `0`.
- **La exportación no infiere la línea base.** Si no se pasó una y no hay una guardada, `baseline_comparison` queda sin deltas y aparece `warning: baseline not selected; baseline_comparison exported without deltas`.
- **Dos PNG opcionales** (`epoch_loss_<ts>.png` y `metric_accuracy_<ts>.png`) se agregan salvo que se use `--no-plots`. Un fallo de `matplotlib` no bloquea los CSV.

10.7 Flujos completos de ejemplo

A. Prueba corta de punta a punta (recomendada para la primera interacción). Usa solo dos configuraciones y tres épocas; tarda poco y permite ver todos los artefactos:

```
python -m prebenchmark generate --experiment-id exp-demo --limit 2

python -m prebenchmark run --experiment-id exp-demo --limit 2 --max-epochs 3 --earlyStop
```

```
# Copiar de la salida el run_id de la primera corrida, por ejemplo:
# run 20261022T143012Z-a1b2c3 config cfg-v1-... seed 42 state COMPLETED epochs 3

python -m prebenchmark compare --experiment-id exp-demo --baseline-run-id 20261022T143012Z-a1b2c3

python -m prebenchmark aggregate --experiment-id exp-demo

python -m prebenchmark export --experiment-id exp-demo --baseline-run-id 20261022T143012Z-a1b2c3 --no-plots
```

En este flujo **no** se puede usar `--documented-default`: las corridas usaron `--max-epochs 3` y `--earlyStop`, así que su `config_id` no es el de la configuración documentada. Por eso la línea base se pasa por `run_id`.

B. Barrido exploratorio multi-semilla. Aquí sí se aprovecha el protocolo de estabilidad, pero el costo crece: 36 configuraciones × 5 semillas = 180 corridas.

```
python -m prebenchmark generate --experiment-id exp-exploratorio

python -m prebenchmark run --experiment-id exp-exploratorio --limit 36 --max-epochs 200 --earlyStop --multi-seed

# Elegir una corrida completada como baseline (copiar su run_id de la salida de run):
python -m prebenchmark compare --experiment-id exp-exploratorio --baseline-run-id 20261022T143012Z-a1b2c3

python -m prebenchmark aggregate --experiment-id exp-exploratorio --grouping config

python -m prebenchmark export --experiment-id exp-exploratorio --baseline-run-id 20261022T143012Z-a1b2c3
```

C. Dos brazos con y sin balanceo de clases. El balanceo no divide el espacio de 36, pero sí cambia el protocolo y el `config_id`. Para que el brazo balanceado exista de verdad, hay que indicar el flag tanto al registrar como al ejecutar:

```
python -m prebenchmark generate --experiment-id exp-plano
python -m prebenchmark run --experiment-id exp-plano --limit 36 --multi-seed

python -m prebenchmark generate --experiment-id exp-balanceado --class-weight balanced
python -m prebenchmark run --experiment-id exp-balanceado --limit 36 --multi-seed --class-weight balanced

python -m prebenchmark aggregate --experiment-id exp-balanceado --grouping config
python -m prebenchmark export --experiment-id exp-balanceado --baseline-run-id <RUN_ID>
```

Para comparar los dos brazos entre sí, `compare` solo mira dentro de un mismo experimento. Si se quieren deltas entre brazos, hay que reunirlos en el mismo `--experiment-id` (por ejemplo, corriendo ambas variantes en el mismo experimento) o analizar los CSV por fuera.

10.8 Errores frecuentes y cómo resolverlos

Mensaje / síntoma	Causa	Solución
`error: baseline must be selected explicitly: pass baseline_run_id, baseline_config_id, or use_documented_default=True`	`compare` se ejecutó sin ninguna opción de línea base.	Pasar una de las tres. La elección nunca es implícita.
`error: pass exactly one of baseline_run_id / baseline_config_id`	Se pasaron dos opciones de línea base a la vez.	Dejar solo una; `--documented-default` tampoco se combina con las otras.
`error: no run found for baseline config_id '...'`	No hay ninguna corrida con ese `config_id`. Es lo habitual con `--documented-default` después de usar `--max-epochs`, `--earlyStop`, otra semilla o `class_weight`.	Usar `--baseline-run-id` con una corrida realmente existente.
`error: baseline run '...' is FAILED; cannot compare against it`	La línea base elegida falló.	Elegir una corrida `COMPLETED` o `COMPLETED_WITH_WARNINGS`.

Mensaje / síntoma	Causa	Solución
`error: no generated configuration has config_id '...'`	El `config_id` no pertenece a la grilla generada con el protocolo por defecto, o se obtuvo de una corrida con overrides.	No usar `--config-id` en ese caso; usar `--limit` o la API programática.
`run` imprime `state FAILED` pero el proceso termina con código `0`.	El runner captura el fallo y lo persiste; el lote continúa.	Revisar `failure` en `metadata.json` o `warnings_summary_<ts>.csv`. El exit code no refleja fallos de corridas.
`warning: baseline not selected; baseline_comparison exported without deltas`	`export` no recibió línea base y no había una guardada en `experiment.json`.	Ejecutar `compare` antes, o pasar `--baseline-run-id` / `--baseline-config-id` / `--documented-default` a `export`.
`export` genera tablas con 0 filas.	No hay corridas en ese `--experiment-id` y `--root`, o el id está mal escrito.	Verificar que exista `results/<EXP>/<run_id>/metadata.json`; revisar `--root`.
`No module named prebenchmark` o `can't open file ...`	Se ejecutó desde otra carpeta o con un Python que no tiene el paquete.	Ejecutar desde la raíz del repositorio y usar el Python del entorno virtual (`.\.venv\Scripts\python.exe`).
El brazo balanceado no aparece en los resultados.	Se pasó `--class-weight balanced` a `generate` pero no a `run`; `run` regenera la grilla sin leer el registro.	Repetir el flag en `run`.

10.9 Referencia rápida

Comando	Propósito	Opciones principales	Escribe
`generate`	Registrar y validar la grilla de candidatos.	`--limit`, `--class-weight`	`configs.jsonl`
`run`	Entrenar configuraciones y persistir cada corrida.	`--limit`, `--config-id`, `--max-epochs`, `--seed`, `--multi-seed`, `--earlyStop`, `--class-weight`	`<run_id>/metadata.json`, `<run_id>/epochs.csv`, `configs.jsonl`
`compare`	Comparar todas las corridas contra una línea base explícita.	`--baseline-run-id` | `--baseline-config-id` | `--documented-default`	`experiment.json` (baseline), `decisions.jsonl`
`aggregate`	Agrupar repeticiones y verificar suficiencia.	`--grouping`, `--min-repetitions`	Nada (solo consola)
`export`	Publicar las tablas CSV (y PNG opcionales).	`--out`, `--no-plots`, opciones de línea base	`<out>/<EXP>/*_<ts>.csv` y PNG opcionales

Secuencia recomendada: `generate` → `run` → `compare` → `aggregate` → `export`. La única dependencia estricta es que `compare` y `export` necesitan corridas ya persistidas; `run` puede ejecutarse sin `generate`, porque regenera la grilla por su cuenta.